Санкт-Петербургский
Федеральный исследовательский центр
Российской академии наук

Санкт-Петербургский
государственный
университет

Технологии СУБД
Практика 2

Прохорова
Анна Александровна

научный сотрудник ЛИТСАМ СПб ФИЦ РАН,
ассистент кафедры информатики СПБГУ

Задачи занятия

Использовать результат SELECT внутри другого запроса.
Записывать один и несколько CTE через WITH.
Различать логические этапы и физическое выполнение.
Сравнивать планы со встраиванием и материализацией.

Вопрос: кто попадёт в TOP-3?

Задача: Вывести трёх очных студентов с наибольшим GPA. При равенстве GPA сначала меньший student_id.

student_id	Имя	GPA	is_full_time
1	Анна	4.00	false
2	Иван	3.90	true
5	Ольга	3.90	true
9	Мария	3.90	true
4	Олег	3.80	true

Какие три имени и в каком порядке вернёт запрос?

Напишите SQL-запрос по условию.
Перед запуском предскажите строки результата.

Учебные данные условные. Сначала сформулируйте ответ без запуска.

[C] PostgreSQL: SELECT

Готовый запрос

Задача: Вывести трёх очных студентов с наибольшим GPA. При равенстве GPA сначала меньший student_id.

```
SELECT student_id, first_name, gpa FROM students
WHERE is_full_time = TRUE
ORDER BY gpa DESC, student_id
LIMIT 3;
```

Сравните со своим решением. Результат и пояснения — на следующем слайде.

Вопрос: кто попадёт в TOP-3? разбор

Задача: Вывести трёх очных студентов с наибольшим GPA. При равенстве GPA сначала меньший student_id.

student_id	first_name	gpa
2	Иван	3.90
5	Ольга	3.90
9	Мария	3.90

Анна исключена условием WHERE: is_full_time = false.
При равном GPA используем student_id по возрастанию.
Олег идёт следующим по GPA и в LIMIT 3 не попадает.

Без второго ключа порядок студентов с GPA 3.90 не был бы задан.

[C] PostgreSQL: SELECT

JOIN: условие соединения

Задача: Вывести первых 10 студентов по student_id вместе с названиями их основных кафедр.

Слева students (студенты), справа departments (кафедры).

```
SELECT s.student_id, s.first_name,
       d.name AS department_name
FROM university.students AS s
JOIN university.departments AS d
  ON d.department_id = s.major_department_id
ORDER BY s.student_id
LIMIT 10;
```

А

Б

А. Исходные строки

s обозначает students. Одна строка здесь соответствует студенту.

Б. Сопоставление

Для каждого студента находим кафедру с тем же идентификатором. INNER JOIN оставляет совпавшие пары.

JOIN добавляет сведения из связанного набора.

[B] PostgreSQL: Table Expressions

Вопрос: одна кафедра и три студента

Задача: Сравнить число строк при INNER JOIN и LEFT JOIN для показанных ниже таблиц.

Слева departments (кафедры), справа students (студенты).

departments — слева

department_id	name
10	Математика
20	Физика

students — справа

student_id	major_department_id
1	10
2	10
3	10

Напишите SQL-запрос по условию.
Перед запуском предскажите строки результата.

Сколько строк даст INNER JOIN? А если заменить его на LEFT JOIN?

[B] PostgreSQL: Table Expressions

Готовый запрос

Задача: Сравнить число строк при INNER JOIN и LEFT JOIN для показанных ниже таблиц.

Слева departments (кафедры), справа students (студенты).

```
SELECT d.name, s.student_id FROM departments AS d
JOIN students AS s ON s.major_department_id = d.department_id;

SELECT d.name, s.student_id FROM departments AS d
LEFT JOIN students AS s ON s.major_department_id = d.department_id;
```

Вопрос: одна кафедра и три студента/ разбор

Задача: Сравнить число строк при INNER JOIN и LEFT JOIN для показанных ниже таблиц.

Слева departments (кафедры), справа students (студенты).

INNER JOIN: 3 строки

Кафедра	student_id
Математика	1
Математика	2
Математика	3

LEFT JOIN: 4 строки

Кафедра	student_id
Математика	1
Математика	2
Математика	3
Физика	NULL

Для кафедры 10 есть три совпадения. У кафедры 20 совпадений нет.

LEFT JOIN сохраняет строку левой таблицы и при отсутствии совпадения.

[B] PostgreSQL: Table Expressions

Вопрос: фильтр после LEFT JOIN

Задача: Вывести пары «кафедра — студент», исключив кафедры без студентов условием WHERE.

Слева departments (кафедры), справа students (студенты).

Данные те же: у кафедры 10 три студента, у кафедры 20 нет студентов.

Напишите SQL-запрос по условию.
Перед запуском предскажите строки результата.

Останется ли кафедра 20? Как изменится результат без WHERE?

[B] PostgreSQL: Table Expressions

Готовый запрос

Задача: Вывести пары «кафедра — студент», исключив кафедры без студентов условием WHERE.

Слева departments (кафедры), справа students (студенты).

```
SELECT d.department_id, s.student_id
FROM departments AS d
LEFT JOIN students AS s
  ON s.major_department_id = d.department_id
WHERE s.student_id IS NOT NULL
ORDER BY d.department_id, s.student_id;
```

Сравните со своим решением. Результат и пояснения — на следующем слайде.

Вопрос: фильтр после LEFT JOIN /разбор

Задача: Вывести пары «кафедра — студент», исключив кафедры без студентов условием WHERE.

Слева departments (кафедры), справа students (студенты).

С WHERE: только три студента кафедры 10

После LEFT JOIN у кафедры 20 значение s.student_id равно NULL.
Условие IS NOT NULL исключает эту строку.

Без WHERE: четыре строки

К трём совпавшим парам добавится кафедра 20 с NULL справа.

Условия WHERE проверяют и строки, полученные внешним соединением.

[B] PostgreSQL: Table Expressions

GROUP BY и HAVING: разные этапы

Задача: Для кафедр с пятью и более студентами вывести число студентов и средний GPA. Сортировка по id кафедры.

```
SELECT major_department_id,
А       COUNT(*) AS students_count,
        AVG(gpa) AS avg_gpa
FROM university.students
Б GROUP BY major_department_id
  HAVING COUNT(*) >= 5
ORDER BY major_department_id;
```

А. Расчёт по группе

Число студентов
и средний GPA
для каждой кафедры.

Б. Группы и отбор

GROUP BY задаёт группы.
HAVING оставляет группы
из пяти и более строк.

Одна строка результата содержит статистику одной кафедры.
*** having работает после группировки в отличие от where**

[B] PostgreSQL: Table Expressions

Вопрос: что посчитает COUNT?

Задача: выведите все кафедры, включая кафедры без студентов. Сравните COUNT(*) и COUNT(s.student_id).

Слева departments (кафедры), справа students (студенты).

Кафедры 10 и 20. У кафедры 10 три студента, у кафедры 20 ни одного.

Напишите SQL-запрос по условию.
Перед запуском предскажите строки результата.

Назовите оба счётчика для каждой кафедры.

[B] PostgreSQL: Table Expressions

Готовый запрос

Задача: выведите все кафедры, включая кафедры без студентов. Сравните COUNT(*) и COUNT(s.student_id).

Слева departments (кафедры), справа students (студенты).

```
SELECT d.department_id,
       COUNT(*) AS joined_rows,
       COUNT(s.student_id) AS students_count
FROM departments AS d
LEFT JOIN students AS s
  ON s.major_department_id = d.department_id
GROUP BY d.department_id
ORDER BY d.department_id;
```

Вопрос: что посчитает COUNT? \разбор

Задача: выведите все кафедры, включая кафедры без студентов. Сравните COUNT(*) и COUNT(s.student_id).

Слева departments (кафедры), справа students (студенты).

department_id	joined_rows	students_count
10	3	3
20	1	0

COUNT(*) считает все строки результата соединения.
COUNT(s.student_id) считает только значения, отличные от NULL.
У кафедры 20 LEFT JOIN сохранил одну строку с NULL справа.

HAVING COUNT(s.student_id) >= 2 оставит только кафедру 10.

[B] PostgreSQL: Table Expressions

Подзапрос: определение

Подзапрос — запрос, вложенный в другую SQL-команду.

```
SELECT student_id, first_name, gpa
FROM university.students
WHERE gpa > (SELECT AVG(gpa) FROM university.students)
ORDER BY gpa DESC, student_id;
```

Внутренний SELECT получает одно среднее значение. Внешний сравнивает GPA каждого студента с этим значением. Это скалярный подзапрос.

Табличный подзапрос

Возвращает набор строк.
В FROM служит источником данных для внешнего запроса.

Скалярный подзапрос

Возвращает один столбец
и не более одной строки.
В выражении даёт одно значение.

[B] PostgreSQL: Table Expressions · [C] PostgreSQL: SELECT

Скалярный подзапрос: средний GPA

Задача: Найти студентов с GPA строго выше среднего по всей таблице. Сначала вывести больший GPA.

```
SELECT student_id, first_name, gpa
FROM university.students
WHERE gpa > (
    SELECT AVG(gpa)
    FROM university.students
)
ORDER BY gpa DESC, student_id;
```

Б

А

А. Одно значение

AVG без GROUP BY вычисляет средний GPA по всей таблице.

Б. Сравнение

GPA каждого студента сравнивается с этим средним значением.

Вложенный SELECT используется здесь как значение в условии.

[C] PostgreSQL: SELECT

Вопрос: GPA выше среднего

Задача: По показанным данным определить студентов с GPA строго выше среднего по всей таблице.

student_id	gpa
1	2.00
2	3.00
3	4.00

Чему равно среднее?
Кто останется
при строгом условии >?

Напишите SQL-запрос по условию.
Перед запуском предскажите строки результата.

Сначала определите значение скалярного подзапроса.

[C] PostgreSQL: SELECT

Готовый запрос

Задача: По показанным данным определить студентов с GPA строго выше среднего по всей таблице.

```
SELECT student_id, gpa FROM students
WHERE gpa > (SELECT AVG(gpa) FROM students)
ORDER BY student_id;
```

Вопрос: GPA выше среднего /разбор

Задача: По показанным данным определить студентов с GPA строго выше среднего по всей таблице.

AVG(gpa) = (2 + 3 + 4) / 3 = 3

student_id	gpa
3	4.00

Студент 2 со значением 3.00 не проходит строгое сравнение > 3.
При замене > на >= в результате будут студенты 2 и 3.

Скалярный подзапрос здесь даёт одно значение для сравнения.

[C] PostgreSQL: SELECT

Подзапрос в FROM: набор строк

Задача: Найти кафедры со средним GPA не ниже 3 с помощью подзапроса в FROM. Сортировка по id кафедры.

```
SELECT stats.major_department_id,
       stats.avg_gpa
FROM (
    SELECT major_department_id,
           AVG(gpa) AS avg_gpa
    FROM university.students
    GROUP BY major_department_id
) AS stats
WHERE stats.avg_gpa >= 3
ORDER BY stats.major_department_id;
```

А

Б

А. Внутренний запрос

Одна строка на кафедру:
её идентификатор
и средний GPA.

Б. Внешний запрос

stats — имя этого набора.
WHERE отбирает строки
с уже рассчитанным avg_gpa.

В PostgreSQL 13 подзапросу в FROM нужен псевдоним.

[B] PostgreSQL: Table Expressions

Два этапа обработки результата

Пример результата внутреннего SELECT

major_department_id	avg_gpa	Условие avg_gpa ≥ 3
1	3.20	Строка остаётся
2	2.80	Строка исключается
3	3.40	Строка остаётся

[B] PostgreSQL: Table Expressions

CTE: общее табличное выражение

CTE = Common Table Expression

CTE — именованная вспомогательная часть SQL-команды, объявленная в WITH.
Имя действует в пределах этой команды.

В этой практике CTE с SELECT описывает набор строк,
к которому обращается основной запрос или следующий CTE.

[A] PostgreSQL: WITH Queries · [C] PostgreSQL: SELECT

WITH: определение и использование CTE

Задача: Найти кафедры со средним GPA не ниже 3 с помощью CTE. Сортировка по id кафедры.

```
А  WITH department_stats AS (
Б      SELECT major_department_id,
             AVG(gpa) AS avg_gpa
      FROM university.students
      GROUP BY major_department_id
  )
В  SELECT major_department_id, avg_gpa
  FROM department_stats
  WHERE avg_gpa >= 3
  ORDER BY major_department_id;
```

А. Имя CTE

department_stats — имя вспомогательного результата.

Б. Определение

В скобках описан SELECT, который вычисляет средний GPA по кафедрам.

В. Использование

Основной SELECT читает CTE и фильтрует его результат.

Выполняется вся команда WITH ... SELECT ...;

[A] PostgreSQL: WITH Queries · [C] PostgreSQL: SELECT

CTE: пояснение

1. Исходные строки students

кафедра 1: GPA 3.0 и 4.0 кафедра 2: GPA 2.0 и 3.0

2. GROUP BY и AVG внутри department_stats

кафедра 1: avg_gpa = 3.5 кафедра 2: avg_gpa = 2.5

3. WHERE avg_gpa >= 3 во внешнем SELECT

В результате остаётся кафедра 1 со средним GPA 3.5.

Числа условные. Показаны логические этапы, а не физический план.

[A] PostgreSQL: WITH Queries · [B] PostgreSQL: Table Expressions

Практика: GPA выше среднего по кафедре

Задача: с помощью CTE найдите студентов, чей GPA выше среднего GPA их кафедры.

student_id	Кафедра	GPA
1	10	2.0
2	10	3.0
3	10	4.0
4	20	3.0
5	20	4.0

Сначала найдите средний GPA для каждой кафедры.

Затем сравните с ним GPA каждого студента.

Напишите запрос через CTE и предскажите результат.

[A] PostgreSQL: WITH Queries · [C] PostgreSQL: SELECT

Решение: CTE и сравнение GPA

Задача: с помощью CTE найдите студентов, чей GPA выше среднего GPA их кафедры.

```
WITH department_stats AS (
    SELECT major_department_id, AVG(gpa) AS avg_gpa
    FROM university.students
    GROUP BY major_department_id
)
SELECT s.student_id, s.gpa, ds.avg_gpa
FROM university.students AS s
JOIN department_stats AS ds
  ON ds.major_department_id = s.major_department_id
WHERE s.gpa > ds.avg_gpa
ORDER BY s.student_id;
```

[A] PostgreSQL: WITH Queries · [C] PostgreSQL: SELECT

Разбор: два этапа вычисления

Задача: с помощью CTE найдите студентов, чей GPA выше среднего GPA их кафедры.

Средние кафедр: 10 — 3.0, 20 — 3.5.

student_id	gpa	avg_gpa кафедры
3	4.0	3.0
5	4.0	3.5

CTE возвращает средний GPA каждой кафедры.
JOIN добавляет нужное среднее к строке студента.
WHERE оставляет только GPA выше этого среднего.

Условные данные. Один этап рассчитывает средние, другой использует их.

[A] PostgreSQL: WITH Queries · [C] PostgreSQL: SELECT

Когда достаточно HAVING

Задача: Найти кафедры со средним GPA не ниже 3, используя GROUP BY и HAVING. Сортировка по id кафедры.

```
SELECT major_department_id,
       AVG(gpa) AS avg_gpa
FROM university.students
GROUP BY major_department_id
HAVING AVG(gpa) >= 3
ORDER BY major_department_id;
```

Тот же результат можно получить без дополнительного уровня запроса.

HAVING фильтрует группы.

Выбираем форму, в которой задача выражена понятнее.

[B] PostgreSQL: Table Expressions

Область действия CTE

Задача: Получить студентов с GPA не ниже 3.5 и проверить, доступно ли имя CTE в следующей команде.

А

```
WITH strong_students AS (
    SELECT student_id, gpa
    FROM university.students
    WHERE gpa >= 3.5
)
SELECT * FROM strong_students;
```

Б

```
SELECT * FROM strong_students;
```

А. Первая команда

WITH и SELECT образуют одну команду. Имя CTE действует внутри неё.

Б. Следующая команда

Определения CTE уже нет. Если отдельного объекта strong_students нет, возникнет ошибка.

Точка с запятой завершает область действия CTE.

[C] PostgreSQL: SELECT

Задача: Предсказать результат двух отдельных команд: чтение CTE в своей команде и после её завершения.

```
WITH strong AS (
    SELECT student_id FROM students WHERE gpa >= 3.5
)
SELECT * FROM strong;

SELECT * FROM strong;
```

Предположим, отдельной таблицы или VIEW strong нет.

Что произойдёт, если выполнить обе команды по очереди?

[A] PostgreSQL: WITH Queries

разбор

Задача: Предсказать результат двух отдельных команд: чтение CTE в своей команде и после её завершения.

Первая команда

Выполняется: WITH определяет strong для своего SELECT.

Вторая команда

Ошибка relation "strong" does not exist.
Определение CTE закончилось вместе с первой командой.

Открытое соединение не продлевает область действия CTE.

[A] PostgreSQL: WITH Queries

Несколько CTE: зависимость результатов

Задача: Вычислить средний GPA кафедр, отобрать среднее не ниже 3 вторым CTE и упорядочить по GPA.

А
```
WITH stats AS (
    SELECT major_department_id,
           AVG(gpa) AS avg_gpa
    FROM university.students
    GROUP BY major_department_id
```
Б
```
), selected AS (
    SELECT * FROM stats
    WHERE avg_gpa >= 3
)
```
В
```
SELECT * FROM selected
ORDER BY avg_gpa DESC, major_department_id;
```

А. Первый CTE

stats вычисляет среднее по кафедрам.

Б. Второй CTE

selected использует stats. Определения разделены запятой.

В. Основной запрос

Читает selected и задаёт порядок выдачи.

Один WITH, несколько имён и один основной запрос.

[C] PostgreSQL: SELECT

Вопрос: два CTE и два порога

Задача: Найти кафедры, где минимум два студента и средний GPA не ниже 3. Использовать два CTE.

Кафедра 10: GPA 2 и 4. Кафедра 20: GPA 4 у одного студента.

Напишите SQL-запрос по условию.
Перед запуском предскажите строки результата.

Какая кафедра останется? Какими будут n и g?

[A] PostgreSQL: WITH Queries

Готовый запрос

Задача: Найти кафедры, где минимум два студента и средний GPA не ниже 3. Использовать два CTE.

```
WITH stats AS (
    SELECT major_department_id, COUNT(*) AS n, AVG(gpa) AS g
    FROM students GROUP BY major_department_id
), selected AS (
    SELECT * FROM stats WHERE n >= 2 AND g >= 3
)
SELECT * FROM selected ORDER BY major_department_id;
```

Вопрос: два CTE и два порога /разбор

Задача: Найти кафедры, где минимум два студента и средний GPA не ниже 3. Использовать два CTE.

Кафедра	n	g	Оба условия выполнены?
10	2	3.0	Да
20	1	4.0	Нет: n < 2

stats вычисляет обе группы.
selected проверяет два условия одновременно, потому что стоит AND.
Высокого среднего недостаточно, если студентов меньше двух.

В результате одна строка: major_department_id = 10, n = 2, g = 3.0.

[A] PostgreSQL: WITH Queries